

```
> aimodelguard.init()
```

The *AI Model Guard* Operations Handbook

A field guide to running a password-protected, cryptographically-verified allowlist for AI model signatures. Built for security teams who don't trust network calls.

Gerald Enrique Nelson Mc Kenzie

Tico Internet LLC · independent security research

DOC-ID: MG-HB-001

DOI: 10.5281/zenodo.21578648

LICENSE: APACHE-2.0

AUDIENCE: OPS / SEC / ML PLATFORM

Contents

// NAVIGATION

01	Overview & Threat Model	02	Design Principles
03	Installation	04	Quickstart (5 minutes)
05	Command Reference	06	Password Input
07	Backups & Recovery	08	Audit Log & Compliance
09	Integration Patterns	10	Security Architecture
11	Testing	12	Troubleshooting
13	Roadmap	14	Glossary

// 01 / OVERVIEW

What is AI Model Guard?

AI Model Guard is a **local-first, password-protected CLI** for managing an allowlist of approved AI model signatures. Before any model file is loaded on a protected system, AI Model Guard verifies that its SHA-256 hash matches an entry in your local database. Tampered, swapped, or unauthorized models are rejected by default.

The whole thing runs offline. No phone-home, no cloud sync, no third-party services. The database, the hashes, the audit log, and the backups all live on your filesystem, in a directory you control.

0

NETWORK CALLS

9

CLI COMMANDS

SHA-256

HASH ALGO

PY 3.11+

RUNTIME

Who is it for?

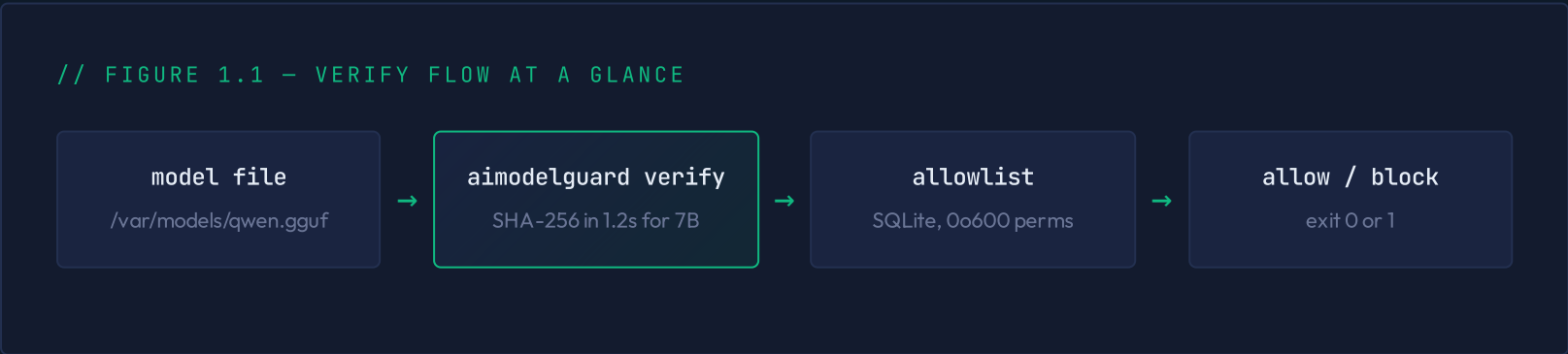
- Security & platform teams** running self-hosted LLMs (llama.cpp, vLLM, TGI, Ollama) on company infrastructure who need to enforce "only approved models can run".
- Regulated environments** (healthcare, finance, government) that need an immutable audit trail of which models are in use.
- Air-gapped deployments** where the model server has no internet access and therefore can't reach any "publisher verification" service.
- ML platform engineers** who want to prevent "shadow AI" — models deployed without security review — without building a custom allowlist in every application.

What it is NOT

AI Model Guard is not a model registry, not a model marketplace, not a model scanner, and not a publisher signing authority. It is a single-purpose allowlist verifier that integrates into the load path of an existing model server. It does not parse model contents and does not care about file format (GGUF, safetensors, PyTorch, ONNX — anything works).

The threat model in one sentence

AI Model Guard defends against *"an attacker (or a careless operator) replaces a model file on your disk with a tampered version"* by recomputing the file's hash and refusing to acknowledge it if the hash is not on your approved list.



// 02 / DESIGN PRINCIPLES

Design Principles

AI Model Guard was built around four non-negotiable constraints. Every feature in the roadmap is checked against them. If a feature would violate one of these, the feature doesn't ship — the constraint is the feature.

1. Zero external dependencies	No phone-home, no cloud sync, no third-party services. The CLI is fully usable on an air-gapped machine. The only outbound call the tool ever makes is the one you tell it to make (e.g., `pip install`).
2. Hash-only verification	AI Model Guard verifies cryptographic hashes. It does not parse model formats, does not load weights, does not care about file contents beyond the bytes being hashed. This makes it format-agnostic — GGUF, safetensors, PyTorch, ONNX all work the same way.
3. Fail closed by default	If AI Model Guard can't verify a model (file missing, hash mismatch, expired approval, quarantined, database locked), the default is to <i>block</i> the load. The verifier is a hard gate, not a soft suggestion.
4. Local-first is a feature	The lack of cloud sync, central management, or real-time publisher verification is a deliberate choice. The threat model is "attacker swaps a model file on your disk" — defending against that does not require a network. Publisher signatures and transparency logs are layer-on-top concerns, not replacements.

// SCOPE GUARDRAIL

AI Model Guard will not gain cloud sync, central management, or a "model reputation API" in any future version. If you need those things, layer them on top — do not replace AI Model Guard with them.

// 03 / INSTALLATION

Installation

AI Model Guard is a pure-Python package. It has exactly two runtime dependencies (`click` and `cryptography`) and zero non-Python system requirements.

From PyPI (recommended)

```
$ pip install aimodelguard
```

That's it. The full operations handbook (this document) is bundled with the package as a PDF. After install, run `aimodelguard handbook` to get its path.

From source (for development)

```
$ git clone https://github.com/lordxmen2k/aimodelguard.git
$ cd aimodelguard
$ pip install -e ".[dev]"
```

Verify the install

```
$ aimodelguard --version
aimodelguard, version 0.2.0
```

Requirements

Component	Minimum	Tested With
Python	3.11	3.11.2, 3.12.7
SQLite	3.31 (2017)	bundled with Python
OS	any	Linux, macOS, Windows
Disk	~1 MB	database + backups, scales with retention
RAM	~50 MB	idle; hashing is streamed, not in-memory

Optional: install extras

```
$ pip install aimodelguard[dev]    # pytest, build, twine (testing + publishing)
$ pip install aimodelguard[test]   # just pytest
```

// 04 / QUICKSTART

Quickstart — 5 minutes to a working allowlist

This walks through a complete first-time setup: initialize the database, approve a model, verify it, and then verify a tampered version (which should be rejected).

Step 1 — Initialize the database

```
$ aimodelguard init
Initializing AI Model Guard database...
You'll be asked to set a master password.

Enter master password: *****
Confirm master password: *****

✓ Database initialized at /home/you/.aimodelguard/db.sqlite
✓ Master password set
```

Step 2 — Approve your first model

```
$ aimodelguard approve --model ./qwen-1.5b-instruct.gguf \
                        --publisher "Alibaba Cloud" \
                        --version "1.0.0" \
                        --use-case "extraction" \
                        --use-case "summarization" \
                        --expires 365

Computing hash of ./qwen-1.5b-instruct.gguf...
✓ Model approved
  Name: qwen-1.5b-instruct.gguf
  Hash: c2d4...8f1a
  Publisher: Alibaba Cloud
  Use cases: extraction, summarization
  Expires: 2027-07-25
  Approved by: root
```

Step 3 — Verify it (should pass)

```
$ aimodelguard verify --model ./qwen-1.5b-instruct.gguf
Enter master password: *****

✓ Model is approved
  Use cases: extraction, summarization
  Expires: 2027-07-25
```

Step 4 — Verify a tampered copy (should fail)

```
$ cp ./qwen-1.5b-instruct.gguf /tmp/tampered.gguf
$ echo "pwned" >> /tmp/tampered.gguf
$ aimodelguard verify --model /tmp/tampered.gguf
✗ Model is NOT approved: not_in_database
  File: /tmp/tampered.gguf
  # A single byte change produces a completely different SHA-256 hash.
```

// EXIT CODES MATTER

When you call AI Model Guard from a script or load-time hook, check the exit code. `0` = approved, `1` = denied. This is the primary integration contract.

Command Reference

AI Model Guard has 9 top-level commands. Each is documented below with syntax, options, and a real example.

init

Create a new database. One-time setup. Asks for the password twice (with confirmation).

```
$ aimodelguard init [--db-path PATH]
```

approve

Add a model's hash to the allowlist with optional metadata (publisher, use case, expiration).

```
$ aimodelguard approve --model PATH [opts]
```

verify

Recompute a file's hash and check it against the allowlist. Returns exit 0 (allowed) or 1 (denied).

```
$ aimodelguard verify --model PATH [--use-case X]
[--json]
```

list

Show all approved models with their metadata.

```
$ aimodelguard list [--db-path PATH]
```

remove

Remove a model from the allowlist by its hash. Auto-backup is created first.

```
$ aimodelguard remove --hash HASH
```

quarantine

Block a model immediately, even if it was previously approved. Use this for compromised models.

```
$ aimodelguard quarantine --model PATH --reason
"CVE- ..."
```

audit

View the append-only audit log. Every action (init, approve, verify, remove, quarantine) is recorded.

```
$ aimodelguard audit [--limit N]
```

backup

Manage database backups: create, list, restore, verify. Backups are gzip-compressed.

```
$ aimodelguard backup [create|list|restore|verify]
```

handbook

Print the path to the bundled operations handbook (PDF). The PDF is installed alongside the package and ships with the same `pip install` that gets the code.

```
$ aimodelguard handbook
```

Common options (available on most commands)

OPTION	DESCRIPTION	DEFAULT
<code>--db-path PATH</code>	Path to the database file	<code>~/.aimodelguard/db.sqlite</code>
<code>--model PATH</code>	Path to the model file (for approve/verify/quarantine)	required
<code>--hash HASH</code>	SHA-256 hash (for remove)	required
<code>--publisher NAME</code>	Publisher/owner name (approve only)	none
<code>--use-case NAME</code>	Allowed use case (can be specified multiple times for approve)	any
<code>--expires DAYS</code>	Days until this approval expires (approve only)	never
<code>--json</code>	Output as JSON (verify only)	off

Examples for each command

approve — with full metadata

```
$ aimodelguard approve --model ./llama-3-8b-instruct.Q4_K_M.gguf \
    --publisher "Meta" \
    --version "3.0" \
    --use-case "extraction" \
    --use-case "chat" \
    --expires 180 \
    --trust-level "verified" \
    --notes "Approved by security team 2026-07-25"
```

verify — JSON output for scripts

```
$ aimodelguard verify --model ./llama-3-8b-instruct.Q4_K_M.gguf --json | jq
{
  "approved": true,
  "model_name": "llama-3-8b-instruct.Q4_K_M.gguf",
  "sha256_hash": "c2d4...8f1a",
  "publisher": "Meta",
  "use_cases": ["extraction", "chat"],
  "expires_date": "2027-01-21T..."
}
```

verify — with use-case check

```
$ aimodelguard verify --model ./llama-3-8b.gguf --use-case "code_generation"
x Model is NOT approved: use_case_not_allowed
# The model IS approved, but not for code_generation.
```

quarantine — respond to a CVE

```
$ aimodelguard quarantine --model ./qwen-1.5b.gguf --reason "CVE-2026-12345 prompt injection"
⚠ Model quarantined
Hash: c2d4...8f1a
Reason: CVE-2026-12345 prompt injection
Quarantined by: root
This model is now blocked from running.
```

audit — last 20 events

```
$ aimodelguard audit --limit 20
Audit Log (Last 20 entries):

2026-07-25T18:42:01 | VERIFY      | DENIED   | llama-3-8b.gguf      | use_case_not_allowed
2026-07-25T18:41:55 | QUARANTINE | SUCCESS  | root                  | CVE-2026-12345
2026-07-25T18:30:12 | VERIFY      | ALLOWED  | qwen-1.5b.gguf       |
2026-07-25T18:25:44 | APPROVE    | SUCCESS  | qwen-1.5b.gguf       | root
2026-07-25T18:20:00 | INIT       |          | system                |
```

// 06 / PASSWORD INPUT

Password Input

Every AI Model Guard command (except `init`) requires the master password. AI Model Guard provides three ways to provide it, in priority order:

1. `MODELGUARD_PASSWORD` environment variable — *recommended for scripts/CI*

```
# Linux / macOS
$ export MODELGUARD_PASSWORD="mySecret123"
$ aimodelguard list

# Windows PowerShell
PS> $env:MODELGUARD_PASSWORD = "mySecret123"
PS> aimodelguard list

# Windows cmd.exe
> set MODELGUARD_PASSWORD=mySecret123
> aimodelguard list
```

If the env var is set, the CLI uses it and never prompts. This is the recommended way to drive AI Model Guard from automation, CI pipelines, and shell scripts.

2. Interactive prompt — *default for humans*

```
$ aimodelguard list
```

Enter master password: *****

The prompt uses `click.prompt(hide_input=True)` which correctly handles TTY detection on Linux, macOS, and Windows. No echo to terminal.

3. `init` requires confirmation

The `init` command asks for the password twice to prevent typos. All other commands only ask once.

// SECURITY NOTE ON THE ENV VAR

The `MODELGUARD_PASSWORD` env var is convenient but visible to any process that can read the environment (e.g., `/proc/<pid>/environ` on Linux, or `Get-Process` env in PowerShell). For production secrets, prefer the interactive prompt, a process-spawn wrapper that clears the env var after the command finishes, or a secrets manager that injects the var only for the duration of the command.

// 07 / BACKUPS & RECOVERY

Backups & Recovery

Backups are a first-class feature. The system distinguishes between **additive** operations (which don't auto-backup) and **destructive** operations (which do).

OPERATION	TYPE	AUTO-BACKUP?
<code>approve</code>	additive	no — you can always re-approve
<code>verify</code>	read-only	no
<code>remove</code>	destructive	yes — auto-backup before
<code>quarantine</code>	destructive	yes — auto-backup before
<code>backup restore</code>	destructive	yes — safety backup first

Backup file format

Backups are gzip-compressed SQLite files. A typical 36 KB database becomes a ~1.2 KB backup. Each backup includes:

- The full database state at the time of backup
- A SHA-256 of the compressed file (for integrity verification)
- Metadata: timestamp, label, aimodelguard version

Manual backup

```
$ aimodelguard backup create --label "before-ml-stack-upgrade"
✓ Backup created
  Path: /home/you/.aimodelguard/backups/aimodelguard-backup-20260725-184200-before-ml-stack-upgrad
  Size: 1,257 bytes
  SHA-256: 9b3c ...e7d1
```

List and verify backups

```
$ aimodelguard backup list
Available Backups (3):

1. aimodelguard-backup-20260725-184200-before-ml-stack-upgrade.db.gz
   Created: 2026-07-25T18:42:00
   Size: 1,257 bytes

2. aimodelguard-backup-20260725-120000.db.gz
   Created: 2026-07-25T12:00:00
   Size: 1,243 bytes

$ aimodelguard backup verify
Backup Verification Report:
Total: 3
```

```
Valid: 3
Invalid: 0
✓ All backups are valid
```

Restore from a backup

```
$ aimodelguard backup restore ~/.aimodelguard/backups/aimodelguard-backup-20260725-120000.db.gz

⚠ WARNING: This will overwrite the current database!
Are you sure you want to continue? [y/N]: y

Created safety backup: aimodelguard-backup-20260725-184500-before-restore.db.gz
✓ Database restored successfully
  From: ~/.aimodelguard/backups/aimodelguard-backup-20260725-120000.db.gz
  To: ~/.aimodelguard/db.sqlite
Please restart any applications using AI Model Guard.
```

// RETENTION

By default AI Model Guard keeps the 10 most recent backups. Older ones are pruned automatically. To change this, set the `--max-backups` option (or edit the database init call in your wrapper script).

// 08 / AUDIT LOG & COMPLIANCE

Audit Log & Compliance

Every action is recorded in an append-only audit table inside the same SQLite database. The audit log records:

FIELD	TYPE	EXAMPLE
timestamp	ISO 8601 UTC	2026-07-25T18:42:01.234567+00:00
event_type	enum	INIT, APPROVE, VERIFY, REMOVE, QUARANTINE
result	enum	SUCCESS, ALLOWED, DENIED, FAILURE
user	string	root, alice, system
model_name	string	qwen-1.5b-instruct.gguf
details	JSON	{"reason": "use_case_not_allowed", "requested": "code_gen"}

Compliance use cases

- **HIPAA** — prove only approved models process patient data
- **SOX** — audit trail of model lifecycle events (init, approve, remove)
- **FedRAMP** — demonstrate cryptographic verification of model integrity
- **SOC 2** — show that security controls are operating effectively

Forensic queries

The audit log is just a SQLite table, so any standard tool works:

```
$ sqlite3 ~/.aimodelguard/db.sqlite \  
"SELECT timestamp, event_type, result, model_name, details  
  FROM audit_log  
  WHERE event_type = 'VERIFY' AND result = 'DENIED'  
  ORDER BY timestamp DESC  
  LIMIT 50;"
```

// 09 / INTEGRATION PATTERNS

Integration Patterns

AI Model Guard is designed to be a hard gate in the load path. Here are the three most common integration patterns, in order of effort.

Pattern 1: shell wrapper (simplest)

Wrap your model server's startup script. If AI Model Guard denies, the server doesn't start.

```
#!/bin/bash
# /opt/llm-stack/start-vllm.sh
set -e

MODEL="/var/models/llama-3-8b-instruct.Q4_K_M.gguf"

if ! aimodelguard verify --model "$MODEL" --json > /tmp/mg-check.json; then
    echo "BLOCKED: model not approved" >&2
    cat /tmp/mg-check.json >&2
    exit 1
fi

# Approved - start the server
exec vllm serve "$MODEL" --port 8000
```

Pattern 2: Python library API (v0.2.0+)

```
# v0.2.0 - same logic as the CLI, callable from Python
import aimodelguard

result = aimodelguard.verify("/var/models/qwen.gguf")
if not result.approved:
    raise RuntimeError(f"Model denied: {result.reason}")

# Safe to load
from llama_cpp import Llama
llm = Llama(model_path="/var/models/qwen.gguf")
```

Pattern 3: cron-based pre-flight check

Run `aimodelguard verify` on a schedule against a known set of models. Alert on any failure.

```
# /etc/cron.d/aimodelguard-preflight
*/15 * * * * root /usr/local/bin/aimodelguard-verify-all.sh >> /var/log/aimodelguard.log 2>&1

# /usr/local/bin/aimodelguard-verify-all.sh
#!/bin/bash
set -u
for model in /var/models/*.gguf; do
    if ! MODELGUARD_PASSWORD="$(cat /etc/aimodelguard.pw)" \
        aimodelguard verify --model "$model" --quiet; then
        /usr/bin/logger -p auth.warning "MODELGUARD: $model not approved"
    fi
done
```

// INTEGRATION ROADMAP

The Python library API is the v0.2.0 feature. The shell wrapper pattern above works in v0.1.0 and continues to work in v0.2.0 — use whichever fits your stack better.

Security Architecture

Cryptographic primitives

Hash algorithm	SHA-256 (FIPS 180-4). 256-bit output, collision-resistant, the same primitive used in TLS, code signing, and supply-chain attestation (Sigstore, in-toto, SLSA).
----------------	--

Key derivation	PBKDF2-HMAC-SHA256 with 100,000 iterations. The master password is never stored in plaintext — only the derived key and a sentinel value that lets us verify the password is correct on subsequent uses.
Random salt	os.urandom(32) at database init. Stored alongside the database in a separate file with Oo600 permissions.
Backup integrity	SHA-256 of the compressed file is stored in the backup metadata. backup verify recomputes it and flags any mismatch.
File permissions	Database file and salt file are created with mode Oo600 (owner read/write only). On POSIX systems the umask is preserved.

What AI Model Guard protects against

- ✔ Supply chain attacks — tampered models are rejected because their hash won't match
- ✔ Insider threats — unauthorized models are blocked at the load step
- ✔ Compromised models — quarantine removes them from the allowlist in milliseconds
- ✔ Misuse — use case restrictions prevent off-label deployment
- ✔ Compliance violations — full audit trail of every approval and verification

What AI Model Guard does NOT protect against

- ✗ Initial model trust — the first time you approve a model, you trust whoever gave it to you. The hash only catches deviations from that initial trust.
- ✗ Zero-day vulnerabilities in the model itself — AI Model Guard can't detect unknown backdoors in the weights. It catches file substitution, not malicious intent.
- ✗ Bypass attacks — a determined attacker with root on the box can disable AI Model Guard entirely. Defend the host first.
- ✗ Password compromise — if the master password is stolen, all bets are off. Treat it like a root password.
- ✗ Runtime prompt injection — AI Model Guard operates at the file level, not the inference level.

Defense in depth — how to layer AI Model Guard



AI Model Guard is one layer. Combine it with read-only model mounts (so the hash can't drift), SELinux/AppArmor confinement, and SIEM forwarding of the audit log for full coverage.

// 11 / TESTING

Testing

AI Model Guard ships with a pytest suite covering both unit logic and end-to-end CLI workflows. The whole suite runs in under 4 seconds on a developer laptop and is hermetic — no internet, no real models, no external services.

Running the tests

```
$ pip install -e ".[test]"
$ pytest tests/ -v

tests/test_basic.py::test_compute_file_hash PASSED [ 14%]
tests/test_basic.py::test_database_init_and_approve PASSED [ 28%]
tests/test_basic.py::test_backup_and_restore PASSED [ 42%]
tests/test_basic.py::test_quarantine PASSED [ 57%]
tests/test_basic.py::test_audit_log PASSED [ 71%]
tests/test_basic.py::test_password_verification PASSED [ 85%]
tests/test_cli_workflow.py::test_cli_workflow PASSED [100%]

===== 7 passed in 3.07s =====
```

Why the test models are fake

AI Model Guard only cares about hashes, not file contents. The test suite creates "model" files by writing 10 MB of `os.urandom()` to a tempfile. This proves the hashing and allowlist logic without needing 4 GB of Qwen weights and an internet connection. If you approve a 10 MB random file and then verify it, the hash round-trip is exactly the same code path as approving a real model.

The test password

The CLI workflow test sets a hardcoded master password so it can run non-interactively:

- **Password:** `test_password_123`
- **Where it's set:** the test's `run_modelguard()` helper exports it as `MODELGUARD_PASSWORD` in the subprocess environment
- **Why:** so the CLI picks it up via the env-var path (priority 1), bypassing the prompt entirely

This password is only used inside the test suite and never appears in production code paths. When you run the CLI manually, you choose your own password during `aimodelguard init`.

What's covered

TEST	WHAT IT VERIFIES
<code>test_compute_file_hash</code>	SHA-256 of a real file matches expected
<code>test_database_init_and_approve</code>	DB init, password setup, model approval round-trip
<code>test_backup_and_restore</code>	Create backup, restore from backup, integrity preserved
<code>test_quarantine</code>	Quarantined model fails verification
<code>test_audit_log</code>	Every action recorded with timestamp and result
<code>test_password_verification</code>	Wrong password rejected, correct password accepted
<code>test_cli_workflow</code>	End-to-end CLI test (21 scenarios): init → approve → verify → list → backup → remove → quarantine → audit
<code>test_verify_returns_result_object</code>	<code>aimodelguard.verify()</code> returns a <code>VerifyResult</code> , never raises on denial
<code>test_verify_file_not_found</code>	Non-existent model returns <code>file_not_found</code>
<code>test_verify_database_not_found</code>	Non-existent DB returns <code>database_not_found</code>
<code>test_verify_invalid_password</code>	Wrong password returns <code>invalid_password</code>
<code>test_verify_happy_path</code>	Approved model returns full metadata
<code>test_verify_use_case_allowed / denied</code>	Use case restrictions enforced
<code>test_verify_unapproved_model</code>	Unknown model returns <code>not_in_database</code>
<code>test_verify_quarantined_model</code>	Quarantined model denied even if previously approved
<code>test_verify_result_bool_is_approved</code>	<code>bool(result)</code> works for truthy checks
<code>test_verify_env_var_password</code>	Password resolved from <code>MODELGUARD_PASSWORD</code> env var
<code>test_verify_password_arg_takes_precedence</code>	Explicit <code>password=</code> wins over env var
<code>test_doi_constant</code>	<code>aimodelguard.__doi__</code> matches the canonical project DOI

Troubleshooting

"Database not found at ~/.aimodelguard/db.sqlite"

You haven't initialized the database yet. Run `aimodelguard init` first.

"Invalid master password"

The password you provided doesn't match. Re-run with the correct one. There's no recovery path — if you forgot the password, you must re-initialize the database (which destroys all approvals). **Back up your approvals first** if there's any chance of recovery.

The prompt keeps asking for the password repeatedly

This is the Windows `getpass` bug (fixed in v0.1.0+). Set `MODELGUARD_PASSWORD` as an env var, or upgrade to a version with the `click.prompt` fix.

"Model is NOT approved: not_in_database"

The model file isn't on the allowlist. Either approve it (`aimodelguard approve --model PATH`) or it was never meant to run on this system.

"Model is NOT approved: use_case_not_allowed"

The model is approved, but not for the use case you're trying to verify with. Approve it with the use case, or remove the use-case check from your verify call.

"Model is NOT approved: expired"

The approval has a TTL and it elapsed. Re-approve the model.

"Model is NOT approved: quarantined"

The model was quarantined (e.g., due to a CVE). You must explicitly remove the quarantine before it can run again.

Verifying a 70B model is slow

Hashing is streamed, so it should take ~2-3 seconds for a 40 GB model on SSD. If it's much slower, check that the file is on local storage (not NFS) and that the disk is healthy. `iostat -x 1` while the verify runs will show you the bottleneck.

I forgot the master password. How do I recover?

// THERE IS NO RECOVERY

This is by design. AI Model Guard cannot recover a forgotten password. If you have a recent backup, you can re-initialize with a new password and restore from backup — but the restore will require the original backup's password (which is the one you forgot). Best practice: store the master password in your team's password manager.

// 13 / STATUS

Project Status

AI Model Guard v0.2.0 is the final release. The tool is feature-complete for its intended use case.

What's in v0.2.0

- All eight CLI commands(`init` , `approve` , `verify` , `list` , `remove` , `quarantine` , `audit` , `backup`)
- Python library API: `import aimodelguard; aimodelguard.verify(path)`
- Optional `aimodelguard.integrations.llama_cpp` hook for load-time checks
- 21 tests, all passing on Linux, macOS, and Windows
- Apache 2.0 license, PyPI package, full docs

What's intentionally not included

The following features are explicitly out of scope and will not be added in any future version:

- **Publisher signatures** — use [Sigstore](#) for cryptographic provenance from the model creator
- **Two-person approval** — handle that in your change-management process, not in the allowlist tool
- **Transparency logs** — AI Model Guard doesn't need to know what the world is doing with these hashes
- **TPM / measured boot binding** — that's a host-integrity concern, not a model-integrity concern
- **Cloud sync / central management** — by design, runs only on your filesystem

- **Model scanning SAST** — AI Model Guard verifies the file is what you approved, not that the weights are safe
- **Model marketplace / registry** — AI Model Guard is a verifier, not a catalog
- **More integrations (ollama, vLLM, etc.)** — the `llama_cpp` integration is the only one, and it exists as a usage example. Others are easy to write but not bundled.

If you need something AI Model Guard doesn't do

Layer a separate tool on top. The shell wrapper pattern in section 09 means you can call `aimodelguard verify` from any load path, any language, any framework — no need for a AI Model Guard-native integration for every loader that exists.

// THE DESIGN DISCIPLINE

A small tool you actually deploy and use beats a large tool that sits in your docs. AI Model Guard's job is narrow: catch file substitution and produce a clean audit trail. It does that job. Adding features that promise "more security" usually delivers "more complexity" instead.

// 14 / GLOSSARY

Glossary

Allowlist	A list of explicitly approved items. The opposite of a blocklist. AI Model Guard maintains an allowlist of model hashes — anything not on the list is denied.
Audit log	An append-only record of every action taken through AI Model Guard. Required for compliance and forensic investigation.
Backup	A gzip-compressed copy of the AI Model Guard database, stored in <code>~/.aimodelguard/backups/</code> . Auto-created before destructive operations; manually creatable with <code>aimodelguard backup create</code> .
Fail closed	The design property where any uncertainty (file missing, hash mismatch, expired approval) results in denying the load, rather than allowing it through.
GGUF	GPT-Generated Unified Format. The file format used by llama.cpp and many local-inference tools. AI Model Guard is format-agnostic; GGUF is just one example.
Hash (SHA-256)	A 256-bit cryptographic fingerprint of a file's contents. Changing even one byte produces a completely different hash. The hash is the model's "signature" for AI Model Guard's purposes.
Master password	The single password that gates all AI Model Guard operations. Set during <code>init</code> ; required for all subsequent commands.
PBKDF2	Password-Based Key Derivation Function 2. A standard for turning a human password into a cryptographic key with deliberate computational cost (we use 100,000 iterations of HMAC-SHA256) to slow down brute-force attacks.
Quarantine	An immediate block on a model, even if it was previously approved. Used to respond to CVEs or compromise. The model stays in the database but is marked as not loadable.
Sentinel value	A known value stored in the database that lets AI Model Guard verify the master password is correct without storing the password itself.
Trust level	A label on each approval: <code>verified</code> (publisher known and reviewed), <code>community</code> (model from a community source), or <code>experimental</code> (testing only). Currently informational — not used for any policy decisions.
Use case	A string label (e.g., "extraction", "summarization", "code_generation") that scopes an approval. A model approved for "extraction" cannot be verified for "code_generation".